[LOGO FAKULTAS]
(letakkan logo.png di folder report/ lalu jalankan ulang generate_report.py)

LAPORAN UJIAN TENGAH SEMESTER
PEMROGRAMAN MULTIPLATFORM

APLIKASI PENCATATAN KEUANGAN PRIBADI "HEMATKU"

Implementasi Asynchronous Programming, SQLite, dan Shared Preferences pada Flutter

Disusun oleh:
Naswa Vifda Zalianti
NIM: [ISI NIM]

[ISI NAMA JURUSAN / PROGRAM STUDI]
[ISI NAMA FAKULTAS]
[ISI NAMA PERGURUAN TINGGI]

2026

DAFTAR ISI

DAFTAR ISI **2**

BAB I. PENDAHULUAN **3**

1.1. Latar Belakang dan Gambaran Aplikasi 3

1.2. Fitur Aplikasi 3

BAB II. PEMBAHASAN **4**

2.1. Desain Antarmuka Pengguna (User Interface) 4

2.2. Persiapan Project dan Package 6

2.3. Implementasi Asynchronous Programming 7

2.4. Implementasi Data Persistence 8

A. SQLite 8

B. Shared Preferences 8

C. Tambahan: Read and Write Files 9

2.5. Pengujian dan Hasil Implementasi 9

BAB III. PENUTUP **11**

3.1. Kesimpulan 11

3.2. Saran 11

DAFTAR PUSTAKA **12**

BAB I. PENDAHULUAN

1.1. Latar Belakang dan Gambaran Aplikasi

Pengelolaan keuangan pribadi merupakan kebutuhan sehari-hari, terutama bagi mahasiswa yang mengatur uang saku, biaya kuliah, transportasi, dan kebutuhan lainnya. Pencatatan secara manual sering terlupa, tidak rapi, dan sulit dievaluasi sehingga pengeluaran menjadi tidak terkontrol. Diperlukan aplikasi sederhana yang dapat mencatat pemasukan dan pengeluaran dengan cepat, menampilkan ringkasan kondisi keuangan, serta tetap dapat digunakan tanpa koneksi internet.

Untuk menjawab kebutuhan tersebut dikembangkan aplikasi **HematKu**, yaitu aplikasi pencatatan keuangan pribadi berbasis Flutter dan Dart. Tujuan pengembangannya adalah (1) menerapkan konsep *asynchronous programming*, (2) menerapkan dua teknik *data persistence*, yaitu SQLite dan Shared Preferences, serta (3) merancang antarmuka yang mudah digunakan. Sasaran pengguna adalah mahasiswa dan masyarakat umum yang ingin mencatat dan memantau keuangan harian. Seluruh data disimpan pada perangkat sehingga aplikasi berfungsi secara *offline*.

1.2. Fitur Aplikasi

No.	Fitur	Fungsi dan Manfaat
1	Splash dan Login	Pengguna masuk dengan email dan password; opsi "Ingat saya" menyimpan sesi sehingga pengguna langsung masuk pada pembukaan berikutnya.
2	Beranda	Menampilkan total saldo, progres anggaran bulanan, ringkasan pemasukan/pengeluaran hari ini, dan transaksi terbaru.
3	Tambah, ubah, hapus transaksi	Mencatat nominal, kategori, tanggal, dan catatan; data dapat diedit atau dihapus kapan saja.
4	Daftar transaksi	Transaksi dikelompokkan per tanggal, dapat difilter (semua/pemasukan/pengeluaran) dan dicari.
5	Pilih kategori	Sembilan kategori bergambar untuk mempercepat input.
6	Grafik	Diagram donut pengeluaran per kategori berdasarkan rentang minggu, bulan, atau tahun.
7	Anggaran bulanan	Pengguna menetapkan batas anggaran dan memantau sisa anggaran.
8	Pengaturan dan profil	Tema terang/gelap, informasi akun, ringkasan total pemasukan dan pengeluaran, serta keluar akun.
9	Laporan transaksi	Mengekspor seluruh transaksi ke berkas PDF atau JSON pada penyimpanan internal perangkat.

BAB II. PEMBAHASAN

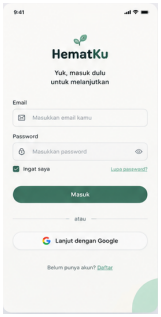
2.1. Desain Antarmuka Pengguna (User Interface)

Antarmuka dirancang dengan Material 3 dengan palet warna hijau (#2F6B4F) sebagai warna utama, latar krem lembut, serta kartu berwarna mint untuk pemasukan dan merah muda untuk pengeluaran agar mudah dibedakan. Navigasi utama menggunakan *bottom navigation bar* dengan empat menu: Beranda, Transaksi, Grafik, dan Profil. Fungsi setiap halaman dijelaskan berikut.

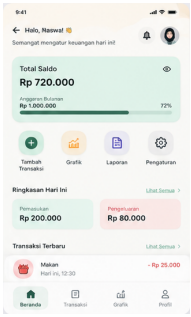
- **Splash (Gambar 2.1)**: halaman pembuka berisi logo, slogan, dan tombol "Mulai".
- **Login (Gambar 2.2)**: formulir email, password dengan tombol tampil/sembunyi, kotak "Ingat saya", tombol Masuk, dan tombol Google.
- **Beranda (Gambar 2.3)**: kartu saldo dan anggaran, menu cepat (Tambah Transaksi, Grafik, Laporan, Pengaturan), ringkasan hari ini, dan transaksi terbaru.
- **Transaksi (Gambar 2.4)**: daftar transaksi per tanggal dengan filter, kolom pencarian, dan tombol tambah (+).
- **Tambah Transaksi (Gambar 2.5)** dan **Pilih Kategori (Gambar 2.6)**: formulir input dan kisi kategori.
- **Edit Transaksi (Gambar 2.7)**: formulir yang sama dengan data terisi serta tombol Hapus Transaksi.
- **Grafik (Gambar 2.8)**: diagram donut dan rincian persentase per kategori.
- **Anggaran Bulanan (Gambar 2.9)**: total anggaran, persentase pemakaian, sisa anggaran, dan tombol Atur Anggaran.
- **Pengaturan (Gambar 2.10)**, **Profil (Gambar 2.11)**, dan **Laporan Transaksi (Gambar 2.12)**.



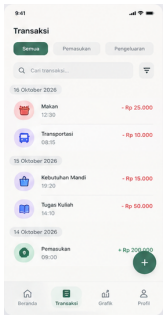
Gambar 2.1 Halaman Splash



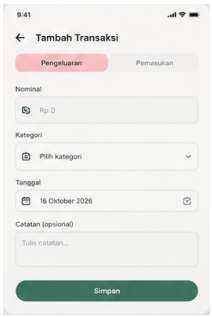
Gambar 2.2 Halaman Login



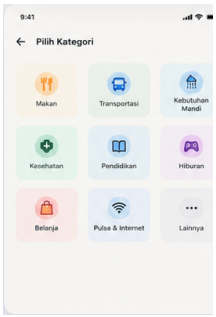
Gambar 2.3 Halaman Beranda



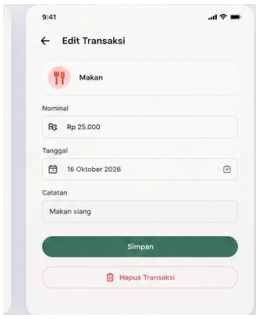
Gambar 2.4 Halaman Daftar Transaksi



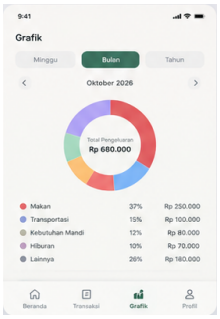
Gambar 2.5 Halaman Tambah Transaksi



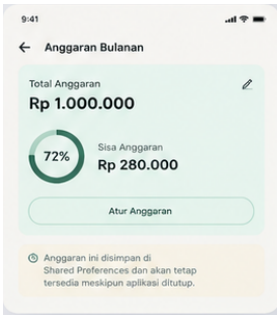
Gambar 2.6 Halaman Pilih Kategori



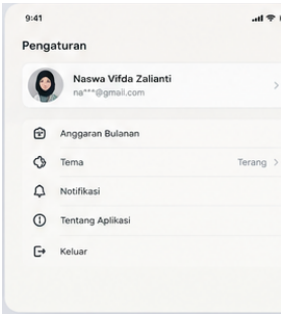
Gambar 2.7 Halaman Edit Transaksi



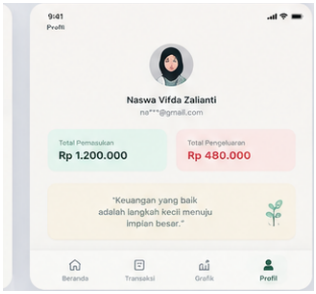
Gambar 2.8 Halaman Grafik



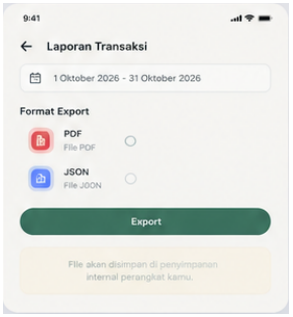
Gambar 2.9 Halaman Anggaran Bulanan



Gambar 2.10 Halaman Pengaturan



Gambar 2.11 Halaman Profil



Gambar 2.12 Halaman Laporan Transaksi

Catatan: gambar di atas adalah rancangan antarmuka (UI design) yang menjadi acuan implementasi. Ganti dengan tangkapan layar aplikasi yang sedang berjalan bila diperlukan.

2.2. Persiapan Project dan Package

Project dibuat dengan perintah `flutter create hematku`, kemudian dikembangkan dengan struktur folder berikut.

```
hematku/
■■■ pubspec.yaml            # dependensi
■■■ README.md
■■■ lib/
    ■■■ main.dart           # entry point, tema, ThemeMode
    ■■■ core.dart           # warna, format rupiah/tanggal, Store (state)
    ■■■ data.dart           # barrel export models & services
    ■■■ models/
    ■    ■■■ category.dart   # data kategori + ikon
    ■    ■■■ transaction.dart# model Tx (toMap/fromMap)
    ■■■ services/
    ■    ■■■ database_service.dart # SQLite + ekspor berkas
    ■    ■■■ prefs_service.dart    # Shared Preferences
    ■■■ screens/            # splash, login, shell, home, transaksi,
                           # form, chart, profile, settings, budget, report
```

Package tambahan yang digunakan:

Package	Fungsi	Alasan Penggunaan
sqflite	Basis data SQLite lokal	Menyimpan data transaksi yang terstruktur dan membutuhkan operasi CRUD.
path	Penyusunan lokasi berkas	Membentuk path basis data dan berkas ekspor lintas platform.
shared_preferences	Penyimpanan key-value	Menyimpan anggaran, tema, email, dan status "Ingat saya".
path_provider	Akses direktori aplikasi	Mendapatkan folder dokumen untuk menulis berkas laporan.
pdf	Pembuatan dokumen PDF	Menghasilkan laporan transaksi berformat PDF.

2.3. Implementasi Asynchronous Programming

Operasi yang memerlukan waktu, seperti akses basis data, pembacaan preferensi, penulisan berkas, dan proses login, dijalankan secara asinkron agar antarmuka tidak membeku. Dart menyediakan `Future` untuk mewakili hasil yang belum tersedia, serta `async` dan `await` untuk menunggu hasil tanpa memblokir *main thread*.

- **Proses login** (login_page.dart): simulasi autentikasi dengan `Future.delayed`; selama menunggu tombol dinonaktifkan dan indikator putar tampil.
- **Memuat transaksi** (core.dart, `Store.load`): membaca SQLite; selama proses `loading=true` aplikasi menampilkan `CircularProgressIndicator`.
- **Menyimpan dan menghapus** transaksi: menunggu operasi basis data selesai, lalu memuat ulang data sehingga UI selalu mutakhir.
- **Ekspor laporan** (report_page.dart): menulis berkas PDF/JSON secara asinkron, lalu menampilkan lokasi berkas.

Kode 1. Proses login asinkron

```
Future<void> masuk() async {
  setState(() => busy = true);                          // tampilkan loading
  await Future.delayed(const Duration(seconds: 1));    // proses butuh waktu
  await Prefs.setEmail(email.text);                     // simpan ke prefs
  await Prefs.setLoggedIn(remember);
  if (!mounted) return;
  Navigator.pushReplacement(context,
      MaterialPageRoute(builder: (_) => const Shell()));
}
```

Fungsi bertanda `async` mengembalikan `Future<void>`. Kata kunci `await` menjeda eksekusi fungsi sampai proses selesai tanpa menghentikan penggambaran UI. Pemeriksaan `mounted` memastikan widget masih aktif sebelum berpindah halaman.

Kode 2. Memuat dan menyimpan data dengan state loading

```
class Store extends ChangeNotifier {
  List<Tx> list = [];
  bool loading = true;
  Future<void> load() async {
    loading = true;  notifyListeners();
    list = await Repo.all();        // async: baca SQLite
    budget = Prefs.budget;
    loading = false; notifyListeners();
  }
  Future<void> save(Tx t) async {
    t.id == null ? await Repo.insert(t) : await Repo.update(t);
    await load();                   // muat ulang setelah simpan
  }
}
```

Pada `main()`, `Prefs.load()` dan `store.load()` di-*await* sebelum `runApp`. Widget `ListenableBuilder` pada halaman utama memantau `Store`: saat `loading` bernilai true tampil indikator, dan setelah selesai data ditampilkan.

Kode 3. Ekspor laporan asinkron

```
Future<void> export() async {
  setState(() => busy = true);
  final f = fmt == 'pdf'
      ? await Repo.exportPdf(store.list)
      : await Repo.exportJson(store.list);   // tulis berkas
  setState(() { busy = false; path = f.path; });
}
```

2.4. Implementasi Data Persistence

Aplikasi menerapkan dua teknik penyimpanan data utama, yaitu **SQLite** dan **Shared Preferences**. Selain itu, fitur laporan menulis berkas PDF/JSON sebagai fitur tambahan.

A. SQLite

- **Alasan pemilihan**: transaksi adalah data terstruktur dalam jumlah banyak yang perlu diurutkan, dicari, diubah, dan dihapus; hal ini sesuai dengan basis data relasional.
- **Jenis data**: tabel `tx` dengan kolom id, expense (0/1), amount, cat, date (ISO-8601), dan note.
- **Hubungan dengan fitur**: dipakai oleh halaman Beranda, Transaksi, Tambah/Edit, Grafik, dan Profil.

```
static Future<Database> get db async => _db ??= await openDatabase(
  join(await getDatabasesPath(), 'hematku.db'), version: 1,
  onCreate: (d, _) => d.execute(
    'CREATE TABLE tx(id INTEGER PRIMARY KEY AUTOINCREMENT, '
    'expense INTEGER, amount INTEGER, cat TEXT, date TEXT, note TEXT)'));

static Future<List<Tx>> all() async =>
  (await (await db).query('tx', orderBy: 'date DESC, id DESC'))
      .map(Tx.fromMap).toList();                        // READ
static Future<void> insert(Tx t) async =>
  (await db).insert('tx', t.toMap()..remove('id'));    // CREATE
static Future<void> update(Tx t) async => (await db)
  .update('tx', t.toMap(), where: 'id=?', whereArgs: [t.id]); // UPDATE
static Future<void> delete(int id) async =>
  (await db).delete('tx', where: 'id=?', whereArgs: [id]);    // DELETE
```

Basis data dibuka satu kali (*lazy singleton*) dan tabel dibuat pada `onCreate`. Setiap operasi CRUD bersifat asinkron. Penyimpanan terjadi saat tombol Simpan ditekan (`Store.save`), pembacaan terjadi saat aplikasi dibuka dan setiap setelah perubahan data.

B. Shared Preferences

- **Alasan pemilihan**: data berupa pengaturan kecil berpasangan kunci-nilai yang tidak perlu struktur tabel.
- **Jenis data**: `budget` (int), `dark` (bool, tema), `loggedIn` (bool, ingat saya), dan `email` (string).
- **Hubungan dengan fitur**: Anggaran Bulanan, Tema pada Pengaturan, "Ingat saya" pada Login, dan email pada Profil.

```
class Prefs {
  static late SharedPreferences _p;
  static Future<void> load() async {
    _p = await SharedPreferences.getInstance();
    dark.value = _p.getBool('dark') ?? false;
  }
  static int get budget => _p.getInt('budget') ?? 1000000;   // READ
  static Future<void> setBudget(int v) => _p.setInt('budget', v); // WRITE
  static Future<void> setDark(bool v) async {
    dark.value = v;                       // ubah tema langsung
    await _p.setBool('dark', v);          // simpan permanen
  }
}
```

Nilai tema disimpan dalam `ValueNotifier` sehingga `MaterialApp` langsung berganti tema saat saklar diubah dan pilihan tetap tersimpan setelah aplikasi ditutup.

C. Tambahan: Read and Write Files

Fitur Laporan menulis seluruh transaksi ke berkas `laporan_hematku.pdf` atau `laporan_hematku.json` di direktori dokumen aplikasi menggunakan `path_provider` dan `dart:io`.

```
static Future<File> exportJson(List<Tx> list) async {
  final dir = await getApplicationDocumentsDirectory();
  final f = File(join(dir.path, 'laporan_hematku.json'));
  return f.writeAsString(const JsonEncoder.withIndent('  ')
      .convert(list.map((e) => e.toMap()).toList()));
}
```

2.5. Pengujian dan Hasil Implementasi

Pengujian dilakukan secara *black-box* pada emulator/perangkat Android dengan menjalankan `flutter run`. Kolom hasil pengujian diisi setelah pengujian dilakukan pada perangkat penguji.

No.	Fitur yang Diuji	Langkah Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Fitur utama: tambah transaksi	Buka Tambah Transaksi, isi nominal 25000, kategori Makan, tanggal, lalu tekan Simpan.	Transaksi muncul di Beranda dan daftar Transaksi; saldo ikut berubah.	[ ] Berhasil [ ] Gagal

No.	Fitur yang Diuji	Langkah Pengujian	Hasil yang Diharapkan	Hasil Pengujian
2	Fitur utama: edit dan hapus	Buka salah satu transaksi, ubah nominal lalu Simpan; ulangi dan tekan Hapus Transaksi.	Data berubah sesuai input; setelah dihapus transaksi hilang dari daftar.	[ ] Berhasil [ ] Gagal
3	Fitur utama: filter, cari, grafik	Gunakan filter Pemasukan/Pengeluaran, ketik kata kunci, dan buka tab Grafik.	Daftar tersaring sesuai input; grafik menampilkan persentase per kategori.	[ ] Berhasil [ ] Gagal
4	Proses asinkron: login	Isi email valid dan password minimal 4 karakter, tekan Masuk.	Muncul indikator loading sekitar 1 detik, lalu pindah ke Beranda.	[ ] Berhasil [ ] Gagal
5	Proses asinkron: muat data	Buka aplikasi dengan beberapa data tersimpan.	Indikator loading tampil singkat, lalu data muncul tanpa UI membeku.	[ ] Berhasil [ ] Gagal
6	Penyimpanan data pertama (SQLite)	Simpan beberapa transaksi, pindah antar menu.	Data tersimpan dan terbaca kembali pada daftar.	[ ] Berhasil [ ] Gagal
7	Penyimpanan data kedua (Shared Preferences)	Atur anggaran menjadi 2.000.000 dan aktifkan Tema Gelap.	Anggaran dan tema berubah dan dapat dibaca kembali.	[ ] Berhasil [ ] Gagal
8	Persistensi data	Tutup aplikasi sepenuhnya lalu buka kembali.	Transaksi, anggaran, tema, dan sesi login (jika "Ingat saya") tetap ada.	[ ] Berhasil [ ] Gagal
9	Mode offline	Aktifkan mode pesawat, gunakan seluruh fitur.	Semua fitur tetap berjalan tanpa internet.	[ ] Berhasil [ ] Gagal
10	Ekspor laporan (berkas)	Buka Laporan, pilih PDF lalu Export; ulangi dengan JSON.	Berkas dibuat dan lokasi berkas ditampilkan.	[ ] Berhasil [ ] Gagal

Kendala dan penanganan. (1) Plugin `sqflite` hanya berjalan pada Android, iOS, dan macOS sehingga pengujian dilakukan pada emulator/perangkat Android, bukan pada peramban web. (2) Pemanggilan `Navigator` setelah `await` dapat menimbulkan peringatan *use_build_context_synchronously*; kendala ini diatasi dengan memeriksa `mounted` sebelum berpindah halaman. (3) Tambahkan di sini kendala lain yang ditemukan saat pengujian, jika ada.

BAB III. PENUTUP

3.1. Kesimpulan

Aplikasi HematKu berhasil dikembangkan menggunakan Flutter dan Dart sebagai aplikasi pencatatan keuangan pribadi dengan fitur pencatatan transaksi, ringkasan saldo, anggaran bulanan, grafik, dan ekspor laporan. Konsep *asynchronous programming* diterapkan dengan `Future`, `async`, dan `await` pada proses login, akses basis data, dan penulisan berkas sehingga antarmuka tetap responsif. Dua teknik *data persistence* berfungsi sesuai kebutuhan: SQLite untuk data transaksi yang terstruktur dan Shared Preferences untuk pengaturan sederhana. Dengan demikian data tetap tersedia setelah aplikasi ditutup dan tanpa koneksi internet, sehingga tujuan pengembangan tercapai.

3.2. Saran

- Menambahkan autentikasi sungguhan (misalnya Firebase Auth) dan sinkronisasi data ke *cloud*.
- Mengamankan data sensitif dengan enkripsi basis data atau *secure storage*.
- Menambahkan transaksi berulang, pengingat anggaran, serta anggaran per kategori.
- Menambahkan grafik tren (garis/batang) dan fitur impor data dari berkas.
- Menambahkan pengujian otomatis (unit dan widget test) dan dukungan lintas platform lebih luas.

DAFTAR PUSTAKA

Flutter. (2026). *Flutter documentation*. https://docs.flutter.dev

Flutter. (2026). *Persist data with SQLite*. https://docs.flutter.dev/cookbook/persistence/sqlite

Flutter. (2026). *Store key-value data on disk*. https://docs.flutter.dev/cookbook/persistence/key-value

Flutter. (2026). *Read and write files*. https://docs.flutter.dev/cookbook/persistence/reading-writing-files

Dart. (2026). *Asynchronous programming: futures, async, await*. https://dart.dev/codelabs/async-await

pub.dev. (2026). *sqflite*. https://pub.dev/packages/sqflite

pub.dev. (2026). *shared_preferences*. https://pub.dev/packages/shared_preferences

pub.dev. (2026). *path_provider*. https://pub.dev/packages/path_provider

pub.dev. (2026). *pdf*. https://pub.dev/packages/pdf